

Unknot Recognition: An Exact Implementation and a Quasipolynomial Complexity Audit

17 September 2026

Abstract

The requested end-to-end $n^{O(\log n)}$ implementation was not completed. This archive instead contains a working, exact, exponential-time recognizer using reduced Khovanov homology over $\mathbb{F}_2$, and a concrete $O(E^4)$ terminal boundary-pattern test for a known 3-ball. We specify their input models, algorithms, correctness arguments, costs, and tests. We then identify the geometric operations and quantitative invariants still needed to implement the accelerated hierarchy in the supplied February 2021 notes. The package has no runtime dependencies, no hidden geometric oracles, and no quasipolynomial mode. Neither small-instance timings nor a high-level lexicographic progress counter are presented as a proof of the requested bound.

Contents

1	Scope and status	2
2	What the supplied notes do and do not specify	2
3	Input model and validation	3
4	The exact recognition algorithm	4
5	An implemented hierarchy component: ball boundary patterns	6
6	Tests, reproducibility, and limitations	7
7	The remaining work for the requested algorithm	8

1 Scope and status

Throughout, n is the number of crossings in the *supplied* knot diagram, not the minimum crossing number of its knot type. A conventional encoding uses $O(n \log n)$ bits. Arbitrarily long edge labels add their actual input-reading and normalization costs; these are not free unit-cost integers. The delivered components are as follows.

Component	Status	Complexity analyzed here
Classical one-component PD validation	Implemented	Polynomial
Closed-braid to PD conversion	Implemented	Polynomial
Reduced Khovanov recognition	Implemented	$2^{O(n)}$
Boundary pattern on a known ball	Implemented	$O((E + 1)^4)$
Accelerated hierarchical multisurfaces	Not implemented	No delivered bound
Full Cheeger/Heegaard restart process	Not implemented	No delivered bound

The recognizer is complete as an ordinary decision algorithm: without a budget, and in the usual unbounded-memory computation model, it terminates on every validated input. It is not complete as a response to the additional time-bound requirement. Indeed it explicitly visits 2^n resolutions, so the supplied code has an exponential lower bound as well as an exponential upper bound.

The boundary-pattern component is genuinely part of the notes' strategy, but is not connected to an implemented hierarchy builder. In particular it cannot be used to infer nontriviality by assigning an arbitrary pattern to a sphere. Its ambient-manifold precondition matters.

2 What the supplied notes do and do not specify

The February 2021 slides announce the $n^{O(\log n)}$ theorem and explain a hierarchy construction/simplification strategy. They introduce essential boundary patterns, violating discs, multisurfaces, and Cheeger regions. Their four speed-ups are a polynomial surface-complexity bound, compressed hierarchy encoding, multisurfaces, and Heegaard/Cheeger control of depth [1, physical pages 66–74].

The final flowchart contains operations named

```
BUILD HIERARCHICAL MULTI-SURFACE; CUT ALONG SURFACE;
USE CHEEGER REGION; HAKEN'S LEMMA; WEAKLY REDUCE;
SIMPLIFY MULTI-SURFACE.
```

These are mathematical operations, not implementations supplied by this archive. Assigning them constant cost would hide the main problem.

The 2026 preprint [2, Section 9] explicitly analyzes a number of steps separately from running time because some constituent steps are not specified precisely enough there for time estimates. That section also does not bound its two global parameters. This investigation did not locate a complete executable specification of the accelerated primitives. That search result is not a claim that no other manuscript exists, nor a refutation of the announced theorem.

2.1 A version distinction that must not be erased

The supplied 109-page February file uses a Cheeger-region factor $1/3$ and states logarithmic hierarchy length on physical pages 101–108. A different, 34-page Oxford file uses $1/10$ and

displays $O((\log n)^2)$ on physical page 33 [3]. These statements were checked in their respective versions. They must not be combined silently: substituting a squared-logarithmic depth into the simple digit-count estimate below gives $n^{O((\log n)^2)}$, not the particular bound requested here. This observation alone establishes no contradiction between the full mathematical arguments.

2.2 The precise conditional counting argument

Suppose a hierarchy process has an integer complexity vector of length at most L , each digit lying in $\{0, \dots, G\}$. The word “genus” is not a sufficient implementation specification: the notes themselves allow related pattern-sensitive complexity measures [1, physical pages 54–59].

For fixed-length vectors, the appropriate positional rank is

$$R(a_1, \dots, a_L) = \sum_{i=1}^L a_i (G+1)^{L-i}. \quad (1)$$

The base is $G+1$, since the digit G is allowed. Missing suffix positions can be treated consistently as maximal possible digits during construction, or one can compare completed hierarchies at major simplification checkpoints. A decrease at the first changed position dominates every possible reset of the suffix. Thus fewer than $(G+1)^L$ strict major decreases are possible. This is an arithmetic fact, not an implemented topological invariant.

Proposition 2.1 (Conditional end-to-end cost). *Assume that the geometric algorithm really supplies the following bounds: all recorded states use at most $B(n)$ bits; each primitive costs at most $P(B(n))$ bit operations; each major iteration uses $O(L)$ primitives; at most $A(n)$ outer restarts occur; and each restart has fewer than $(G+1)^L$ major iterations. Then its time is at most*

$$O\left(A(n)L(G+1)^LP(B(n))\right),$$

up to initialization and output costs. If A, B, P, G are polynomially bounded in the required sense and $L = O(\log n)$, this is $n^{O(\log n)}$.

Proof. Multiply the number of outer restarts, iterations per restart, primitives per iteration, and cost per primitive. For $G \leq n^c$ and $L \leq d \log n$, the logarithm of $(G+1)^L$ is $O((\log n)^2)$. Polynomial factors do not change that asymptotic form. Every factor is an explicit hypothesis; none follows merely from writing the flowchart. $\square$

The accelerated hierarchy would require, among other things, a bound on *compressed bit-size*, not on the number of uncompressed normal discs. An exponentially large integer weight can have polynomial bit-length; expanding that many discs loses this advantage. Similarly, a low-genus surface need not have a short uncompressed embedding description. A separate quantitative argument is needed for resets that replace the Heegaard structure rather than decrease the current hierarchy rank.

3 Input model and validation

A planar-diagram crossing is a tuple (a, b, c, d) in counterclockwise cyclic order. The understrand joins a to c and the overstrand joins b to d . Each edge label occurs exactly twice. The program rennumbers labels and makes the selected basepoint edge number zero. Repeated labels within a crossing are allowed when they describe legitimate loop edges.

The validator first checks incidence counts and follows opposite half-edges to count knot components. It rejects anything other than a single component. It then validates planarity rather than

assuming that an arbitrary PD-like code describes a classical knot. Let α pair the two darts of each edge, and let σ cyclically rotate the four darts at each crossing. The cycles of $\sigma\alpha$ are the faces of the associated oriented rotation system. Since the projection graph is connected, it is spherical exactly when

$$V - E + F = n - 2n + F = 2.$$

Inputs failing this test are rejected, not classified as virtual unknots.

The exceptional input `{"pd": []}` denotes one crossing-free circle. It does not encode an empty link or a collection of unlinked circles. These choices remove ambiguities at $n = 0$.

A second input format closes a signed Artin braid. Positive generators send the upper-left strand over the upper-right strand. The underlying strand permutation gives the number of closure components. Fresh labels are assigned below each crossing, bottom positions are identified with their corresponding top positions, and the resulting PD undergoes the same validator. Generator signs are retained: changing a trefoil crossing changes the computed answer in the test suite.

4 The exact recognition algorithm

4.1 The reduced resolution complex

We use the standard Khovanov construction [4, 5], with coefficients in $\mathbb{F}_2$ and with one marked circle constrained to carry x . This is the reduced-subcomplex convention described, for example, in [7, Section 2.2]. Writhe-dependent grading shifts are omitted, because they cannot change the total homology dimension.

For each state $s \in \{0, 1\}^n$, use the smoothings

$$0 : (a, b), (c, d), \quad 1 : (a, d), (b, c).$$

A disjoint-set computation determines the $c(s)$ resolution circles. Order them by their least edge, so the marked circle is always circle zero. Over $A = \mathbb{F}_2[x]/(x^2)$, the reduced summand is

$$C_s = (x) \otimes A^{\otimes(c(s)-1)}, \quad \dim C_s = 2^{c(s)-1}.$$

Place it in cube height $|s|$. A basis vector is encoded by one bit for each unmarked circle: zero means 1, one means x . The marked bit is inserted as one when a saddle map is applied, then removed on output.

An edge $s \rightarrow t$ changes one zero smoothing to one. It either merges two circles or splits one. Unchanged circles carry their labels through unchanged. The affected labels use

$$\begin{aligned} m(1 \otimes 1) &= 1, & m(1 \otimes x) &= x, & m(x \otimes 1) &= x, & m(x \otimes x) &= 0, \\ \Delta(1) &= 1 \otimes x + x \otimes 1, & \Delta(x) &= x \otimes x. \end{aligned}$$

These formulas also show directly that the marked- x subspace is preserved. For example, merging a marked x with another x gives zero; splitting a marked x gives $x \otimes x$.

Cube signs disappear over $\mathbb{F}_2$. The commuting-square identities for these maps imply $d^2 = 0$, because each two-edge route occurs twice. The optional `-check-d2` diagnostic additionally multiplies the actual implemented differentials on every basis vector. It is exhaustive but costs extra time and may store more matrix data than the default streaming mode.

4.2 Exact matrix ranks

For each height h , the program streams columns of $d_h : C_h \rightarrow C_{h+1}$ as Python integer bitsets. Identical target terms cancel by XOR. Gaussian elimination chooses the highest remaining bit as

pivot; an existing pivot is XORed away, and a new pivot is retained. There are no floating-point operations, randomized rank estimates, or truncated words.

Let $D_h = \dim C_h$ and $r_h = \text{rank}_{\mathbb{F}_2} d_h$, with missing endpoint ranks zero. Then

$$\beta_h = D_h - r_h - r_{h-1}, \quad H = \sum_h \beta_h = \sum_h D_h - 2 \sum_h r_h. \quad (2)$$

The answer is **unknot** exactly when $H = 1$. The output reports the individual D_h , r_h , and β_h so this arithmetic is inspectable. The height indices are not asserted to be normalized knot-invariant gradings.

4.3 Why the rank-one test is a decision procedure

Kronheimer and Mrowka prove that a knot is the unknot exactly when its reduced Khovanov cohomology has rational rank one [6]. The use of $\mathbb{F}_2$, rather than $\mathbb{Q}$, requires an argument; it is not an interchangeable coefficient convention without justification.

Lemma 4.1. *For a finite complex of free abelian groups, the total dimension of its homology after reduction modulo two is at least its total rational homology dimension.*

Proof. An integer matrix has rank modulo two at most its rational rank: a minor nonzero modulo two is a nonzero integer. The chain dimensions are the same over both fields. Applying (2) to the two coefficient fields gives the stated inequality. Equivalently, this follows from the universal coefficient theorem. $\square$

Theorem 4.2 (Correctness of the implemented decision rule). *For a validated classical knot diagram D representing K ,*

$$\dim_{\mathbb{F}_2} \widetilde{\text{Kh}}(K; \mathbb{F}_2) = 1 \iff K \text{ is the unknot.}$$

Consequently the unbudgeted recognizer returns the correct answer on every input in its stated domain.

Proof. The reduced integral homology of the unknot is one copy of $\mathbb{Z}$, so its modulo-two dimension is one. Conversely, suppose the modulo-two dimension is one. The rational dimension is at most one by Lemma 4.1 and at least one: the graded Euler characteristic is the normalized Jones polynomial, whose value at one for a knot is one. Omitting the homological grading shift can change its sign, but not its nonvanishing. Thus the rational dimension is exactly one. The Kronheimer–Mrowka theorem gives unknotness. The implemented finite chain construction and exact elimination compute precisely the left side of the equivalence. $\square$

This is a mathematical correctness argument for the specified algorithm, not a formal proof of the Python implementation. The tests below provide software evidence for the latter connection.

4.4 Its actual asymptotic cost

Proposition 4.3. *In a bit-cost model with conventional input labels, the implemented unbudgeted recognizer takes $2^{O(n)}$ time and space. A deliberately coarse upper bound is $\text{poly}(n)64^n$ time and $\text{poly}(n)16^n$ space. It does not satisfy $n^{O(\log n)}$ time.*

Proof. A complete smoothing has at most $n + 1$ circles: starting from the one-component knot, each local reconnection can increase the number of components by at most one. Therefore the total number of chain generators is

$$D = \sum_{s \in \{0,1\}^n} 2^{c(s)-1} \leq 2^n 2^n = 4^n.$$

Each differential has at most D rows and columns. At most D pivot eliminations per column, each operating on at most D bits, give a cubic bit-cost bound, with polynomial factors covering indexing and map generation. The stored bitsets have a quadratic bound. Constructing all resolutions adds only a polynomial factor times 2^n . The exhaustive d^2 diagnostic remains within the same coarse exponential form.

Independently of these upper bounds, the program visits all 2^n states. For arbitrarily large n , this already exceeds every fixed $n^{\log n}$. No benchmarking claim can remove that explicit enumeration. $\square$

5 An implemented hierarchy component: ball boundary patterns

5.1 Domain and encoding

The notes define a boundary pattern as a disjoint union of simple closed curves and trivalent graphs. A proper disc meeting the pattern at most three times is violating unless one of the boundary discs it cuts off contains the empty pattern, an arc, or a tripod [1, physical pages 24–25].

Our terminal tester receives such a pattern on the boundary of a *known 3-ball*. Its JSON must explicitly state `ambient: "3-ball"`. This is an asserted precondition, not something the tester proves. A spherical boundary alone does not identify an arbitrary 3-manifold with a ball.

Each graph vertex stores a cyclic triple of darts; edge e consists of darts $2e$ and $2e + 1$. Loops and parallel edges are supported. The code validates the Euler characteristic of each connected rotation-system component. Additional vertex-free circles are counted separately. Relative nesting of disconnected components is unnecessary for this particular essentiality test.

5.2 Reduction to bonds of size at most three

A *bond* is an inclusion-minimal nonempty edge cut. In a connected plane graph, bonds correspond to simple cycles in the planar dual. In this setting a bond of size r describes a Jordan curve transverse to exactly r pattern edges. Push the disc that it bounds on the sphere slightly into the ball to obtain a proper disc.

Proposition 5.1. *An empty pattern or a single vertex-free circle is essential. Any pattern with at least two connected components is inessential. For a connected cubic graph pattern, essentiality is equivalent to the absence of one-edge and two-edge bonds and the absence of three-edge bonds whose two vertex sides both contain more than one vertex.*

Proof. With no pattern, every boundary disc has an allowed empty side. For a single circle, a transverse Jordan curve meeting it at most three times has zero or two intersections, giving an empty or arc side. Distinct components can be separated by a disjoint Jordan curve with pattern on both sides, giving a zero-intersection violation.

Now let the graph be connected. Its complementary regions are discs, so a transverse boundary curve is represented by a closed walk of length at most three in the dual graph. Removing an immediate backtrack corresponds to removing an inessential bigon. A reduced such walk is

a loop, a two-edge cycle, or a triangle; equivalently it gives a bond of size at most three. A backtrack alone cuts off an allowed arc and is not an obstruction.

For a bond of size r , each of its two vertex sides induces a connected subgraph. A side with k vertices has $(3k - r)/2$ internal edges, and hence cycle rank

$$b_1 = \frac{3k - r}{2} - k + 1 = \frac{k - r + 2}{2}.$$

For $r = 1, 2$, both sides contain a cycle, which excludes an empty pattern, arc, or tripod. For $r = 3$, a side is cycle-free precisely when it is a single vertex; the cut then encloses an allowed tripod. Otherwise both sides contain cycles and the disc is violating. These cases account for all the short dual walks and prove the characterization. $\square$

The implementation enumerates subsets of one, two, and three edges. Removing the subset must leave exactly two connected vertex sets, and the subset must be exactly the set of edges crossing between them. These conditions exclude redundant removed edges. The single-vertex three-edge case is then discarded. For a negative result, the code emits the cut and its two sides; a separate routine checks that witness directly without enumerating other cuts.

Proposition 5.2. *For E graph edges, the terminal pattern decision takes $O((E + 1)^4)$ elementary graph operations, with polynomial bit overhead. A supplied bond witness is checked in $O(E + 1)$ graph operations.*

Proof. There are $O((E + 1)^3)$ candidate subsets. A component traversal and boundary check each cost $O(V + E)$, and for a cubic graph $3V = 2E$. The direct witness check performs only a constant number of these traversals. Input integers and the separate count of vertex-free circles incur their actual bit costs. $\square$

This component implements the terminal short-dual-cycle idea in the notes' last flowchart. It does not discover an appropriate hierarchy, construct a normal surface, or certify the 3-ball precondition.

6 Tests, reproducibility, and limitations

The archived run passed **45 test methods**. The tests include all **160** one-component closures among the four-letter three-braid words on $\sigma_1^{\pm 1}, \sigma_2^{\pm 1}$. Each receives a full differential-square check. Other tests cover type-I curls, braid cancellations for type II, nonvacuous braid-relation tests for type III, Markov stabilization, braid conjugation, mirrors, edge relabeling, crossing reordering, basepoints, nonclassical inputs, and exhausted budgets.

A separate small reference implementation uses graph traversal rather than the production disjoint-set resolution code, and constructs saddle outputs with explicit label tuples. Its matrix ranks are compared with the production complex. A separate list-of-bits Gaussian elimination also checks 144 small random matrices and the zero-dimensional cases. These checks are not claimed to be a formal verification or external-package cross-validation.

Example	Crossings	Resolutions	Generators	$\dim_{\mathbb{F}_2} \widetilde{\text{Kh}}$
Crossing-free unknot	0	1	1	1
Trefoil	3	8	15	3
Figure-eight	4	16	33	5
$T(2, 5)$	5	32	123	5
Kinoshita–Terasaka, up to mirror	11	2048	11559	33
Conway, up to mirror	11	2048	11559	33
Constructed braid-conjugate unknot	12	4096	27789	1

These are actual computed results. The two 11-crossing PDs and their names come from Knot Atlas, which also records their Alexander polynomial as one [8]. Their nontriviality is not inferred from a failed Alexander test; the full reduced complex is computed. The dimension 33 values above are regression outputs of this implementation.

For patterns, examples include the empty pattern, one and two circles, a theta graph, a loop-ended dumbbell, a two-edge-bond example, a tetrahedral graph, a triangular prism, and a cube. The prism produces a violating three-edge cut; the tetrahedral graph’s vertex-isolating three-edge cuts are correctly accepted as tripods. Malformed and nonspherical rotation systems are rejected.

The archive runs with Python 3.9 or newer and was tested with Python 3.13.5. No external knot package was installed or executed for validation. The `reproduce.py` script regenerates reports and the test transcript using only the standard library. The checked-in transcript records the actual interpreter and result; `BENCHMARKS.json` records single-run timings with exhaustive d^2 checking enabled. Such timings are not asymptotic evidence.

6.1 Budgets and what a report proves

A state-count or generator-count limit stops construction before silently truncating the complex. A cooperative deadline is also checked during construction and elimination. Exceeding a configured budget returns `unknown`, never `knotted`. A process may still exhaust physical memory without a configured size guard; this is not a mathematical answer.

The `verify` command reconstructs the stored input and recomputes all algebraic dimensions while checking $d^2 = 0$. This detects altered output data, but it shares the production implementation. It is not a short certificate, a polynomial verifier, or a proof of agreement with a separately supplied original input. The recognizer returns no spanning disc, ambient isotopy, Reidemeister sequence, or formal Lean theorem.

7 The remaining work for the requested algorithm

An accelerated implementation needs explicit, executable versions of the following operations, together with their invariants. The list records work not accomplished in this package, rather than asserting mathematical impossibility or absence from all literature.

An embedded exterior representation. Construct the knot exterior and a layered handle/-Morse structure from the PD, retaining the embedding data in S^3 required by the generalized Seifert-surface constructions. A bare abstract triangulation does not automatically preserve all that auxiliary data.

Compressed cuts and pattern updates. Cut along encoded surfaces without expanding all normal pieces. Track orientability, connected components, boundary curves, and the inherited pattern, including the provenance needed to trace a violating disc back to an earlier hierarchy surface. Prove bounds on the number and bit-length of records after repeated operations.

Hierarchical multisurface construction and simplification. Produce the actual surfaces with the requisite independent relative homology classes, not just their ranks or homology vectors. Preserve the needed pattern conditions, choose valid components after compression, and establish the correct polynomial bound on the *pattern-sensitive* complexity digit.

Cheeger-region processing. Recognize the relevant regions in the encoded generalized Heegaard structure, implement the associated replacement, and measure a quantity that bounds all resulting restarts. Merely testing a numerical inequality on asserted genera does not supply the region or the replacement homeomorphism data.

Haken’s lemma and weak reduction. Construct the required disc pairs and modified handle structures from a violating disc. The operation must come with a size and running-time bound, not a call to an exhaustive normal surface search whose exponential dependence has been suppressed.

A global complexity proof tied to code. Identify the precise rank invariant, logarithmic hierarchy-depth bound, outer-restart bound, and bit-cost of every primitive. Only after these obligations are discharged does the conditional estimate in Section 2 become a proof about a running program.

A terminal graph tester and an exact regression oracle are useful components for developing and testing that implementation. They are not substitutes for these geometric constructions. The delivered archive therefore remains an explicitly partial result on the original time-bounded implementation task.

References

- [1] Marc Lackenby. *Unknot recognition in quasi-polynomial time*. February 2021. User-supplied 109-page slide deck, [quasipolynomial-talk.pdf](#). Physical page references include incremental slide builds.
- [2] Marc Lackenby. *Incompressible surfaces, hierarchies and unknot recognition*. 2026. arXiv:2607.23350v1.
- [3] Marc Lackenby. *Unknot recognition in quasi-polynomial time*. Oxford compressed slides, 34-page version, physical page 33. <https://people.maths.ox.ac.uk/lackenby/quasipolynomial-talk-oxford-compressed.pdf>.
- [4] Mikhail Khovanov. A categorification of the Jones polynomial. *Duke Mathematical Journal* 101 (2000), 359–426. arXiv:math/9908171.
- [5] Dror Bar-Natan. Khovanov’s homology for tangles and cobordisms. *Geometry & Topology* 9 (2005). <https://www.math.utoronto.ca/drorbn/papers/Cobordism/Cobordism.pdf>.
- [6] P. B. Kronheimer and T. S. Mrowka. Khovanov homology is an unknot-detector. *Publications Mathématiques de l’IHÉS* 113 (2011), 97–208. arXiv:1005.4346.
- [7] Andrew Lobb and Raphael Zentner. *On spectral sequences from Khovanov homology*. Section 2.2. https://www.maths.dur.ac.uk/users/andrew.lobb/SS_khov.pdf.
- [8] Knot Atlas. K11n42 and K11n34: PD presentations and polynomial invariants. <https://katlas.org/wiki/K11n42>; <https://katlas.org/wiki/K11n34>. Accessed 17 September 2026.